

Perfect. This is exactly the point where I think we should switch from "discussion" to **engineering documentation**.

I will write this exactly as if I am the **Technical Architect** of the project and your teammates are the developers who will implement it.

Engineering Design Document (EDD-01)

Environment Support Module

Project: Smart Feature Release Management System

Version: 1.0

Priority: Critical

Status: Design Approved

1. Executive Summary

Overview

The Environment Support module introduces deployment environments into the Feature Release Management System. It allows a single feature flag to maintain independent configurations across multiple deployment stages such as Development, Staging, and Production.

Instead of duplicating feature flags for every environment, the system separates **Feature Metadata** from **Environment Configuration**, resulting in a scalable, maintainable, and production-oriented architecture.

2. Problem Statement

Current Situation

Currently, each Feature Flag stores its own rollout percentage, enabled state, and kill switch.

Example

Dark Mode

Enabled : true

Rollout : 25%

This configuration is shared everywhere.

Why is this wrong?

Imagine the development team is working on Dark Mode.

Desired configuration

Environment	Enabled	Rollout
Development	✓	100%
Staging	✓	50%
Production	✓	5%

Our current architecture cannot represent this.

Changing rollout immediately affects Production.

This defeats the purpose of safe deployments.

3. Business Motivation

Modern software companies never deploy directly into Production.

Typical deployment lifecycle

Developer

↓

Development

↓

QA

↓

Staging

↓

Production

Every environment has different users, different testing objectives and different rollout strategies.

Without environment support, Feature Release Management remains incomplete.

4. Goals

The Environment Module must allow

- ✓ Multiple deployment environments per project
- ✓ Independent feature configuration in every environment
- ✓ Future SDK support
- ✓ Future Analytics support
- ✓ Future Cache support
- ✓ Future Deployment History

while avoiding duplicated Feature Flags.

5. Non Goals

This module WILL NOT implement

- ✗ SDK
- ✗ Audit Log
- ✗ Notifications
- ✗ Cache
- ✗ Analytics

✗ Environment Variables

These are separate modules.

6. Existing Architecture

Organization

↓

Project

↓

Feature Flag

Current FeatureFlag

Dark Mode

Enabled

Rollout

Kill Switch

Owner

Description

Problem

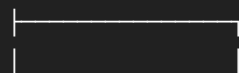
Feature metadata and runtime configuration are stored together.

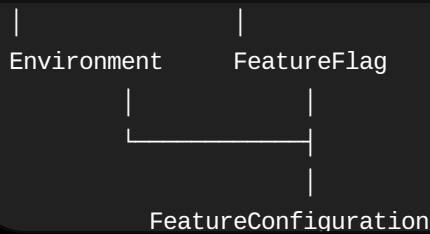
These represent two different concepts.

7. Proposed Architecture

Organization

Project





This architecture separates

FeatureFlag

Stores

- Name
- Key
- Description
- Owner
- Tags
- Metadata

FeatureConfiguration

Stores

- Environment
- Enabled
- Rollout
- Kill Switch
- Rules
- Variables

This separation is the core architectural decision of this module.

8. Why this Design?

There are two possible approaches.

Approach A

Duplicate Feature Flags.

Dark Mode Dev

Dark Mode Stage

Dark Mode Prod

Problems

- duplicate names
- duplicate descriptions
- duplicate owner
- duplicate tags
- synchronization issues

Rejected.

Approach B (Chosen)

Single Feature



Multiple Configurations

Dark Mode



Development Config



Staging Config



Production Config

Advantages

- normalized database
- scalable
- no duplicated metadata
- easy SDK evaluation
- easy analytics
- easy caching

Chosen.

9. Functional Requirements

FR-1

Whenever a Project is created

Automatically create

Development

Staging

Production

No user interaction required.

FR-2

User can create custom environments.

Examples

QA

Testing

Canary

Beta

PreProd

FR-3

User can

Rename Environment

Change Color

Change Display Order

FR-4

Default environments cannot be deleted.

Reason

Prevents accidental removal of Production.

FR-5

Custom environments can be deleted.

FR-6

Whenever a Feature is created

Automatically create FeatureConfiguration for every existing environment.

FR-7

Whenever a new Environment is created

Automatically create FeatureConfiguration for every existing feature.

This guarantees

Feature × Environment

Always has one configuration.

10. User Stories

Story 1

As a Developer

I want Development Environment

So I can safely test new features.

Story 2

As QA

I want Staging

So I can verify before Production.

Story 3

As Product Owner

I want Production rollout

So I can release gradually.

Story 4

As Organization Owner

I want Custom Environments

So I can support our deployment workflow.

11. Technical Decisions

Decision 1

Project automatically creates

Development

Staging

Production

Reason

Zero configuration.

Immediate usability.

Decision 2

Allow unlimited custom environments.

Reason

Every company has different deployment pipelines.

Decision 3

FeatureConfiguration is a separate collection.

Reason

Configuration changes much more frequently than feature metadata.

Separating them improves maintainability.

Decision 4

Every FeatureConfiguration is created immediately.

No lazy creation.

Reason

Avoid null checks

Simpler SDK

Simpler frontend

Simpler backend

Decision 5

Environment comparison uses

environmentId

or

slug

Never compare using display name.

12. Environment Lifecycle

Project Created



Create

Development

Staging

Production



Project Ready

Feature Created



Save Feature



Fetch All Environments



Create Configuration for each



Done

Environment Created



Save Environment



Fetch All Features

↓

Create Configuration for each

↓

Done

Environment Deleted

Allowed only if

Custom Environment

↓

Delete Configurations

↓

Delete Environment

Project Deleted

↓

Delete Environments

↓

Delete FeatureConfigurations

↓

Delete Features

13. Database Design

(We already designed this in detail and will use it as the implementation reference.)

Collections:

- Environment

- FeatureFlag (metadata only)
- FeatureConfiguration (environment-specific state)

Key constraints:

- (projectId, slug) unique for Environment.
- (projectId, key) unique for FeatureFlag.
- (featureId, environmentId) unique for FeatureConfiguration.

14. Permission Matrix

Action	Owner	Admin	Developer	Viewer
View Environments	✓	✓	✓	✓
Create Environment	✓	✓	✗	✗
Rename Environment	✓	✓	✗	✗
Delete Custom Environment	✓	✗	✗	✗
Change Environment Color	✓	✓	✗	✗
Change Rollout	✓	✓	✓	✗

15. API Summary

Environment

- Create
- List
- Update
- Delete (custom only)

Feature Configuration

- Get all configurations for a feature
- Update one environment configuration

The API implementation must remain RESTful and reuse the existing controller
→ service → repository architecture already present in the project.

16. Edge Cases

- Duplicate environment names within the same project.
 - Renaming "Production" should not break SDKs because SDKs use `environmentId / slug`, not the display name.
 - Creating a new environment in a project with hundreds of existing features must generate configurations for all of them.
 - Deleting a custom environment must remove its FeatureConfiguration documents.
 - Default environments cannot be deleted.
 - Environment names may change, but slugs remain unique.
-

17. Performance Considerations

Even with:

- 500 features
- 6 environments

the system stores 3,000 FeatureConfiguration documents.

This is acceptable for MongoDB and greatly simplifies reads and writes.

Optimize with indexes on:

- `featureId`
 - `environmentId`
 - `(featureId, environmentId)`
-

18. Team Responsibilities

Backend Developer

- Environment model
- Repository
- Service
- Controller
- CRUD APIs
- Auto-create default environments
- Auto-create FeatureConfiguration

Frontend Developer

- Environment tabs/switcher
- Create/Edit/Delete dialogs
- Configuration view per environment

QA

- Verify lifecycle flows
 - Validate permissions
 - Test edge cases
-

19. Acceptance Criteria

This module is considered complete only if:

- ☒ New projects automatically contain Development, Staging, and Production.
 - ☒ Users can add custom environments.
 - ☒ Default environments cannot be deleted.
 - ☒ Custom environments can be deleted.
 - ☒ Every feature has exactly one configuration per environment.
 - ☒ Creating a feature creates configurations for all existing environments.
 - ☒ Creating an environment creates configurations for all existing features.
 - ☒ Environment switching correctly loads the corresponding configuration.
 - ☒ All permission rules are enforced.
 - ☒ Unit and integration tests pass.
-

20. Future Scope

This design intentionally leaves room for future modules without schema changes.

Possible additions include:

- Environment-specific SDK keys.
- Deployment history.
- Analytics and metrics.
- Redis caching.
- Environment secrets.
- Webhooks.
- Approval workflows.

Final Architecture Decision

This module is **approved** with the following guiding principle:

A Feature Flag represents the identity of a feature. A Feature Configuration represents how that feature behaves in a specific environment.

This separation avoids duplication, keeps metadata consistent, simplifies SDK evaluation, and provides a scalable foundation for the remaining modules of the Feature Release Management System.

My review of this document

If I were reviewing this as a senior engineer, I'd approve it as a strong **Version 1.0**. Before implementation, the only additional artifacts I'd create are an ER diagram, sequence diagrams, and detailed API request/response contracts. Those are implementation aids rather than architectural decisions, so I consider the core design for the **Environment Support** module complete and ready to hand off to your team.